

Sparse Autoencoder Master Documentation

Online Shopping Behaviour Analysis

Master document covering project summary, theory A-Z, variable/method flow, workflow flowchart, results, BI interpretation, limitations, and glossary.

Document scope

This document covers the Sparse Autoencoder notebook only. It does not cover the dense MNIST, convolutional denoising, or VAE notebooks. The focus here is the Online Shoppers behaviour dataset and the sparse autoencoder business story.

Item	Details
Notebook	sparse_autoencoder.ipynb
Dataset	Online Shoppers Purchasing Intention data
Business area	Digital marketing analytics and customer behaviour analysis
Main model	Sparse Autoencoder with L1 activity regularisation
Main business output	Hidden behaviour segments with different purchase rates and unusual-behaviour levels

Master Document Structure

Section	Purpose
1. Project / Notebook Summary	What the notebook did and why it matters.
2. Theory A-Z	Simple explanation of all important ideas behind sparse autoencoders.
3. Notebook Workflow	Clean top-to-bottom flowchart and step-by-step process.
4. Variable / Method / Data Flow	How variables, methods, and outputs move through the notebook.
5. Results and BI Story	Charts, model comparison, segments, and business interpretation.
6. Limitations and Next Steps	What to improve later and how to extend the work.
7. Glossary	Simple definitions of important terms.

1. Project / Notebook Summary

Project objective

Use a sparse autoencoder to learn compact hidden behaviour features from online shopping sessions, then use those features for business analysis such as segmentation, purchase-pattern comparison, and unusual-session investigation.

1.1 Business Context

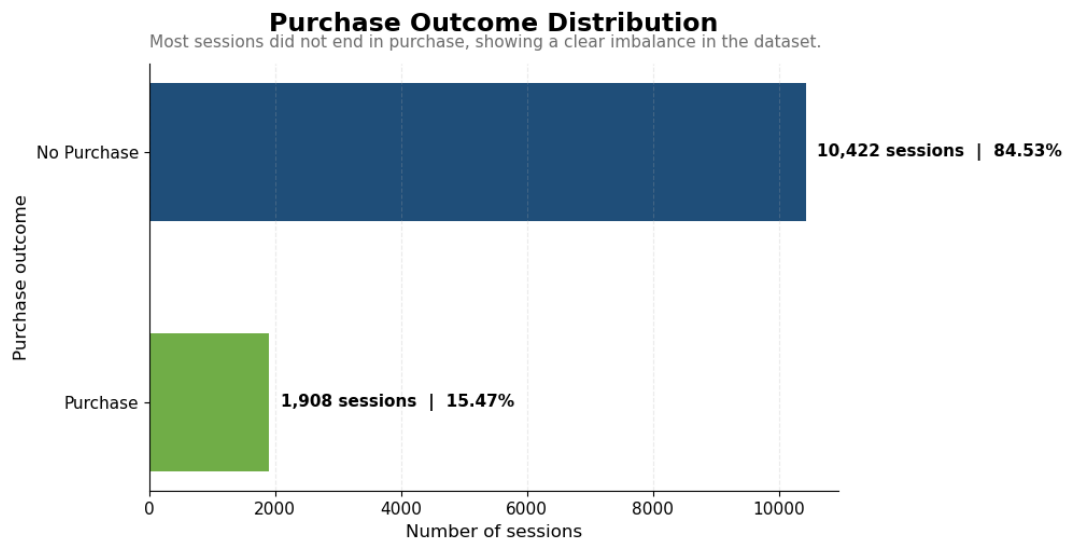
Each row in the dataset represents one online shopping session. The columns describe behaviour such as page visits, time spent on page groups, bounce rate, exit rate, page value, visitor type, weekend visit, and purchase outcome.

The notebook is relevant for digital marketing analytics because it looks at customer behaviour before or without purchase. The aim is not only to build a model, but to turn hidden behaviour patterns into business insight.

Business Question	How the notebook answers it
Can we compress website behaviour into fewer useful signals?	Yes. The sparse encoder compresses 28 processed features into 12 bottleneck values.
Do hidden behaviour groups show different purchase rates?	Yes. Segment purchase rates range from 1.70% to 47.17%.
Can sparse features support segmentation?	Yes. KMeans creates 4 behaviour segments from bottleneck features.
Can hidden features reveal unusual sessions?	Yes. Segment 4 has the highest reconstruction error and is visually separated.
Which segment is most actionable?	Segment 2 combines high purchase rate with useful volume.

1.2 Dataset and Modelling Summary

Topic	Notebook value
Rows and columns	12,330 sessions, 18 original columns
Purchase distribution	No Purchase: 84.53%; Purchase: 15.47%
Original input features	17 columns used as autoencoder input; Revenue kept for interpretation
Processed input features	28 features after scaling and one-hot encoding
Train / Validation / Test split	8,631 / 1,849 / 1,850 rows
Bottleneck size	12 hidden values per session
Segments created	4 hidden behaviour segments



Business note: Purchase sessions are only 15.47% of all sessions. We keep Revenue for interpretation, not as the autoencoder target.

Figure 1. Purchase distribution. The dataset is imbalanced, but Revenue is used for interpretation, not as the autoencoder training target.

1.3 Final Technical Results

Metric	Normal Autoencoder	Sparse Autoencoder	Interpretation
Validation Reconstruction Loss	0.036117	0.045389	Normal AE reconstructs validation data better.
Average Test Reconstruction Error	0.034884	0.043283	Normal AE reconstructs unseen data better.
Average Bottleneck Activation	2.487932	1.762660	Sparse AE uses a quieter bottleneck.
Near-Zero Bottleneck Activations %	0.198305%	0.585902%	Sparse AE pushes more values closer to zero.

Main model trade-off

The normal autoencoder wins on reconstruction accuracy. The sparse autoencoder is more useful for compact hidden features and business interpretation. This is the main trade-off in the notebook.

2. Main Theory A-Z

This section explains the important theory in simple words. The goal is to understand the notebook, not drown in textbook fog. A rare kindness from the machines.

2.1 What Is an Autoencoder?

An autoencoder is a neural network that learns to copy its input. It takes input features, compresses them into a smaller middle layer, and then tries to reconstruct the original input.

In this notebook, one input row is one website session. The model learns to reconstruct that session behaviour.

2.2 Encoder, Bottleneck, and Decoder

The encoder compresses the input. The bottleneck stores the compressed hidden summary. The decoder tries to rebuild the original input from that summary.

For this notebook, the processed session has 28 input features. The bottleneck has 12 values. So the model learns a shorter behaviour summary for each session.

2.3 What Makes It Sparse?

A sparse autoencoder adds a rule: do not use all bottleneck neurons too strongly all the time.

This is done using L1 activity regularisation on the bottleneck layer. It adds a small penalty when bottleneck values are active.

2.4 Reconstruction Error

Reconstruction error measures how far the reconstructed output is from the original input.

Lower error means better reconstruction. Higher error can mean the session is harder to reconstruct or more unusual.

2.5 Sparsity Trade-off

Sparse autoencoders often reconstruct slightly worse than normal autoencoders.

The benefit is that the hidden representation can become simpler and easier to use for later analysis.

2.6 PCA for Visualising Hidden Features

The bottleneck has 12 hidden values. Humans cannot easily see 12 dimensions, because biology took a wrong turn somewhere.

PCA reduces the 12 values into 2 chart dimensions. This lets us visually inspect patterns.

2.7 KMeans for Segmentation

KMeans groups sessions that have similar hidden bottleneck values.

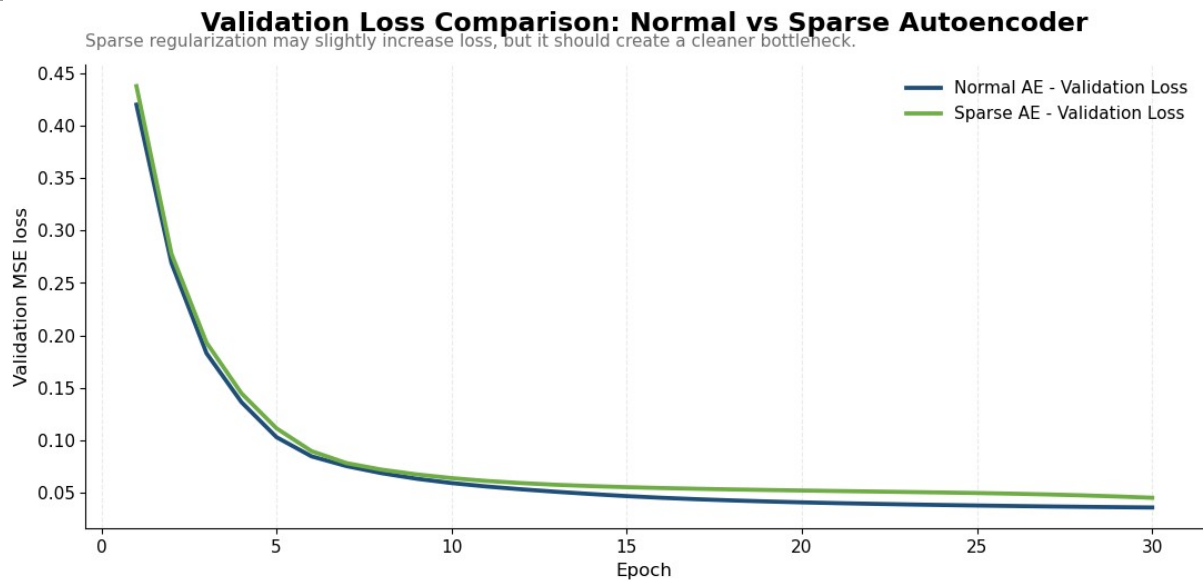
In this notebook, KMeans created 4 behaviour segments from the sparse bottleneck features.

2.8 Theory Summary Table

Concept	Simple meaning	How it was used in this notebook
Autoencoder	Model that learns to reconstruct its input.	Reconstructed online shopping session features.
Encoder	Compresses input data.	Compressed 28 processed features into hidden values.
Bottleneck	Small middle layer.	Stored 12 behaviour values per session.
Decoder	Rebuilds input from bottleneck.	Reconstructed the original 28 processed features.
Sparse autoencoder	Autoencoder encouraged to use fewer active hidden values.	Used L1 activity regularisation in the bottleneck.
L1 activity regularisation	Penalty for too much activation.	Reduced average bottleneck activation.
PCA	Turns many hidden values into 2 visual dimensions.	Visualised bottleneck behaviour space.
KMeans	Groups similar rows.	Created four hidden behaviour segments.
Reconstruction error	Difference between original and rebuilt values.	Used to compare models and identify unusual segments.

2.9 Model Architecture in Simple Form

Model	Structure	Key point
Normal Autoencoder	28 -> 32 -> 12 -> 32 -> 28	Best reconstruction accuracy.
Sparse Autoencoder	28 -> 32 -> 12 -> 32 -> 28 + L1 bottleneck rule	More compact hidden behaviour features.



Business note: Reconstruction loss shows how well each model rebuilds customer-session behaviour features.

Figure 2. Validation loss comparison. The sparse model has slightly higher validation loss, which is expected because it is constrained by sparsity.

3. Clean Top-to-Bottom Flowchart

This section shows the full notebook workflow from data loading to business interpretation. This is the clean process view of the notebook.



Figure 3. Top-to-bottom workflow from raw data to sparse autoencoder business insights.

3.1 Workflow Stage Summary

Stage	Output created
Load and inspect data	Raw session behaviour dataset and purchase distribution.
Prepare features	17 original feature columns and Revenue kept separately.
Split data	Train, validation, and test sets with stable purchase ratio.
Preprocess data	28 processed numeric features ready for neural network input.
Train models	Normal autoencoder and sparse autoencoder.
Compare models	Reconstruction loss and bottleneck sparsity comparison.
Analyse hidden features	PCA view and KMeans behaviour segments.
Create BI story	Purchase rate, segment size, reconstruction error, and opportunity matrix.

4. Variable / Method / Data Flow

This section shows how the important variables and methods connect. The aim is to make the notebook easy to follow from top to bottom.

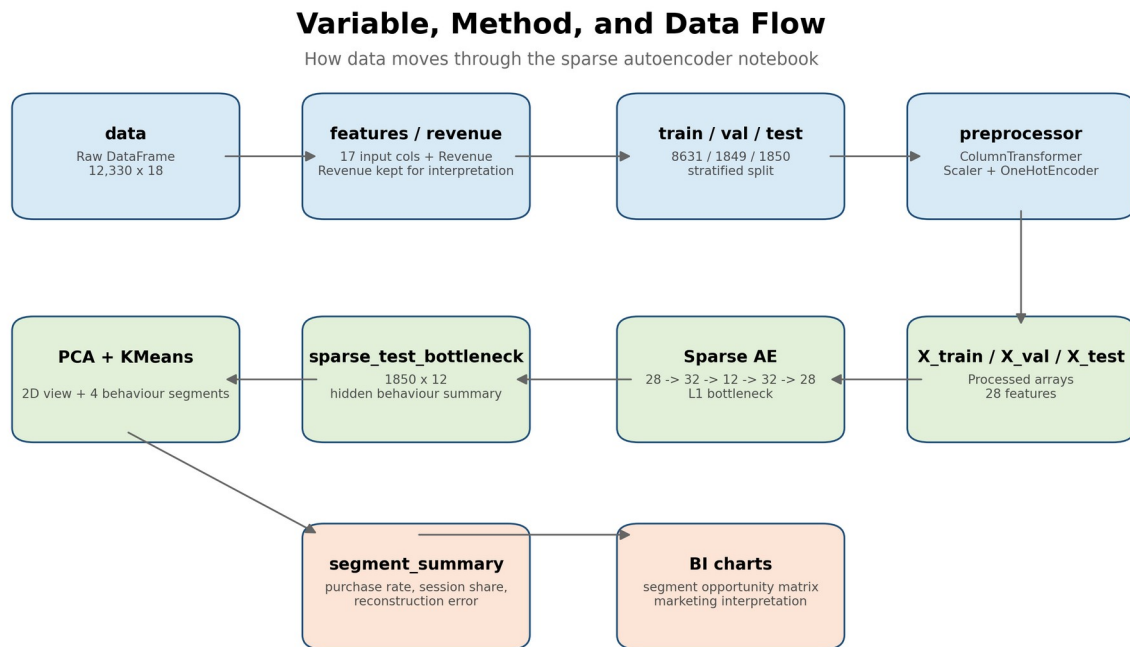


Figure 4. Variable, method, and data flow across the sparse autoencoder notebook.

4.1 Main Variable Flow

Notebook object	Comes from	What it stores	Used for
data	pd.read_csv(data_url)	Raw dataset with 12,330 rows and 18 columns.	Dataset inspection and feature selection.
feature_columns	numeric + categorical + boolean feature columns	17 input columns excluding Revenue.	Inputs for autoencoder.
revenue	data["Revenue"]	Purchase outcome values.	Interpretation and chart colouring only.
train_features / val_features / test_features	train_test_split	Raw features split into train, validation, and test sets.	Training, checking, and final evaluation.
preprocessor	ColumnTransformer	Rules for scaling and one-hot encoding.	Clean numeric input creation.
X_train / X_val / X_test	preprocessor transforms	Processed numeric arrays with 28 features.	Neural network input and reconstruction target.
normal_autoencoder	Keras model	Baseline model without sparsity.	Reconstruction benchmark.
sparse_autoencoder	Keras model with L1 bottleneck	Sparse model with quieter bottleneck.	Compact hidden feature learning.
sparse_test_bottleneck	sparse_encoder.predict(X_test)	12 hidden values for each test session.	PCA, KMeans, business segmentation.
pca_plot_data	PCA + Revenue + segment labels	2D view, Revenue labels, and segments.	Charts and BI storytelling.
segment_summary	groupby Segment Label	Sessions, purchase rate, session share, reconstruction error.	Business insight tables and charts.

4.2 Important Methods and Why They Were Used

Method / Tool	Purpose	Why it matters here
<code>train_test_split</code>	Creates train, validation, and test sets.	Keeps the final test set unseen.
<code>stratify=revenue</code>	Keeps purchase ratio similar across splits.	Important because purchase sessions are only 15.47%.
<code>ColumnTransformer</code>	Applies different preprocessing to different column groups.	Numeric columns need scaling; categorical columns need encoding.
<code>StandardScaler</code>	Scales numeric values.	Prevents large-duration columns from dominating the model.
<code>OneHotEncoder</code>	Turns text categories into numeric columns.	Makes Month and VisitorType usable by neural networks.
Dense layers	Build fully connected neural network layers.	Used for encoder, bottleneck, and decoder.
<code>activity_regularizer=regularizers.l1</code>	Adds sparsity penalty to bottleneck activity.	Main difference between normal and sparse autoencoder.
mean squared error (MSE)	Measures reconstruction error.	Used for training loss and final model comparison.
PCA	Reduces 12 bottleneck values to 2D.	Makes hidden behaviour space visible.
KMeans	Groups similar hidden behaviour patterns.	Creates business-friendly behaviour segments.

5. Results and BI Storytelling

BI storyline

The sparse autoencoder did not beat the normal autoencoder on reconstruction accuracy. Its value came from the hidden bottleneck features, which created behaviour segments with different purchase rates and unusual-behaviour levels.

5.1 Model Learning and Reconstruction

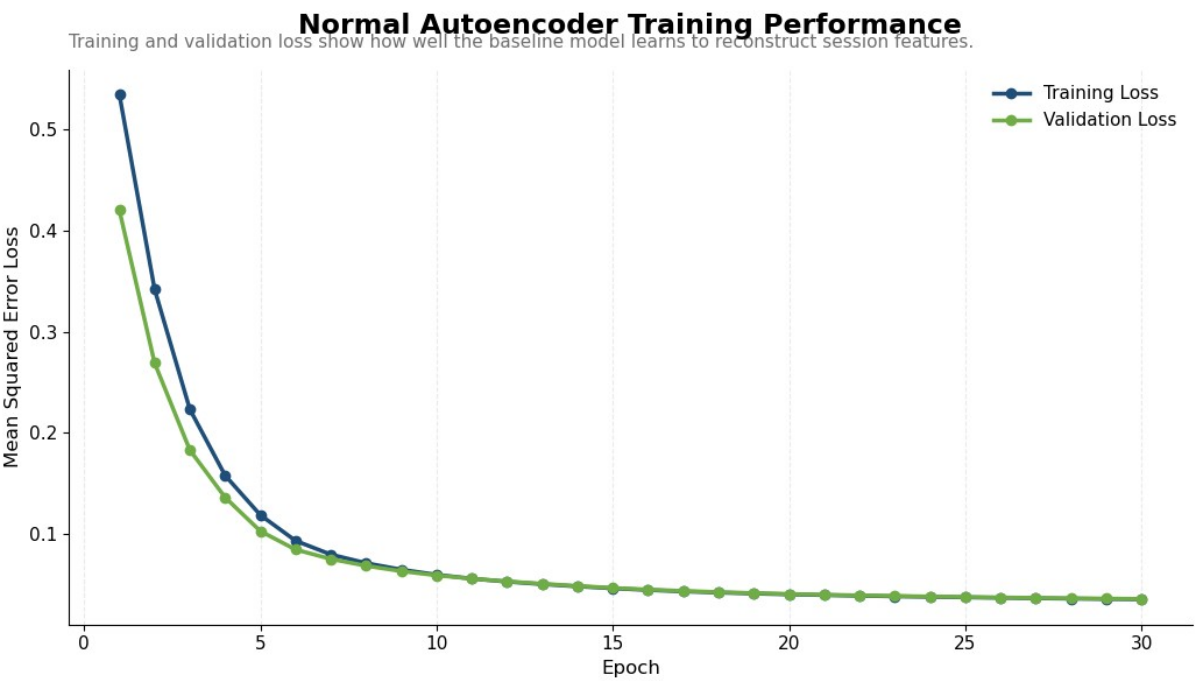
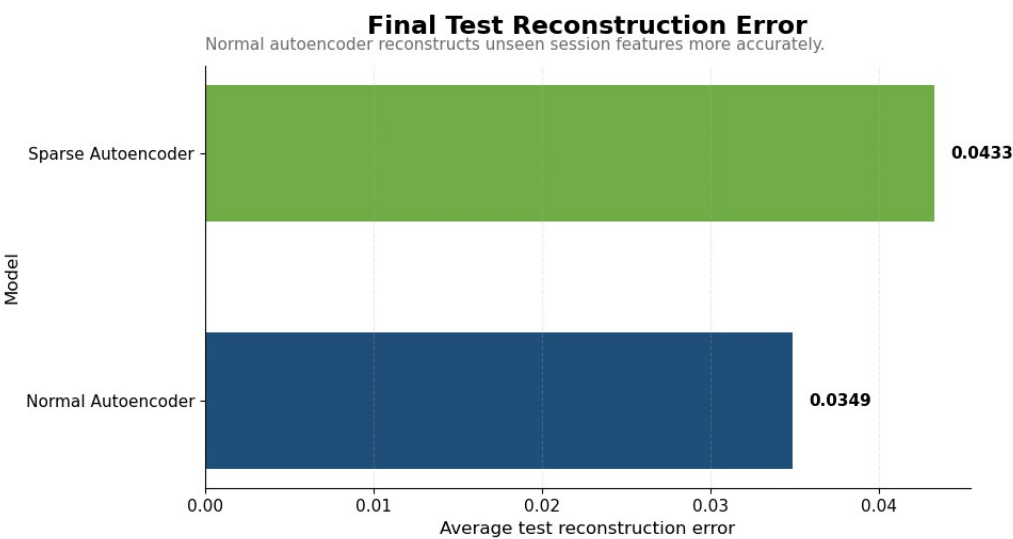


Figure 5. Normal autoencoder training performance. Training and validation losses decrease and stay close, showing stable learning.



Business note: Lower error means better reconstruction. Sparse AE trades some accuracy for a simpler hidden representation.

Figure 6. Final test reconstruction error. The normal autoencoder reconstructs unseen sessions more accurately.

Insight	Meaning
Normal AE has lower error	It is better for pure reconstruction.
Sparse AE has higher error	It sacrifices some reconstruction accuracy for a simpler hidden representation.
This is expected	The sparse bottleneck has an extra rule that discourages high activation.

5.2 Bottleneck Compactness

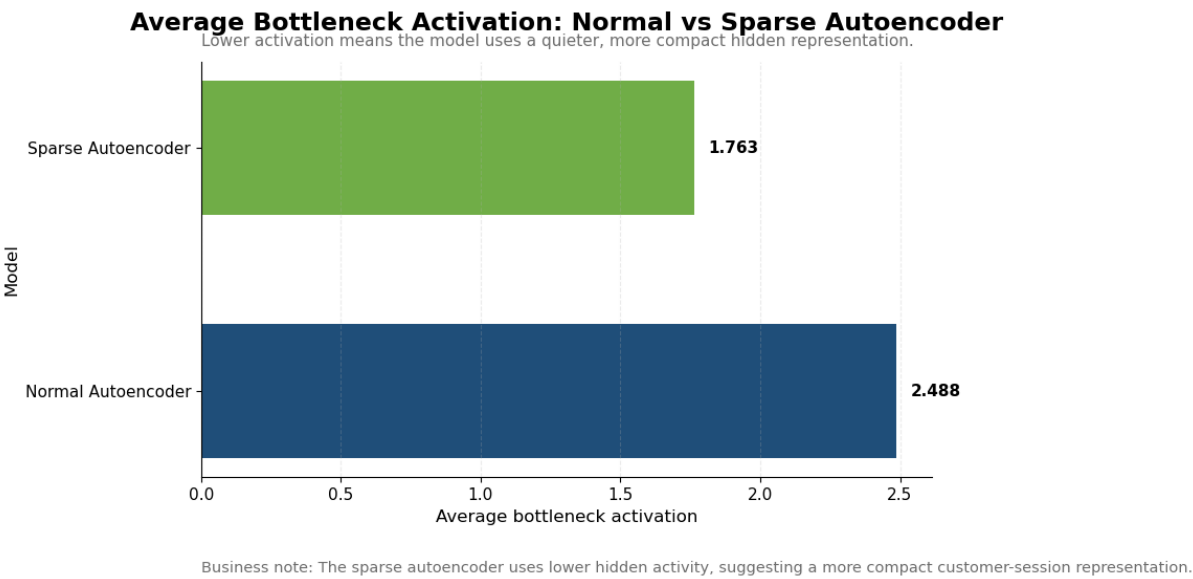


Figure 7. Average bottleneck activation. Sparse AE uses lower hidden activity than the normal AE.

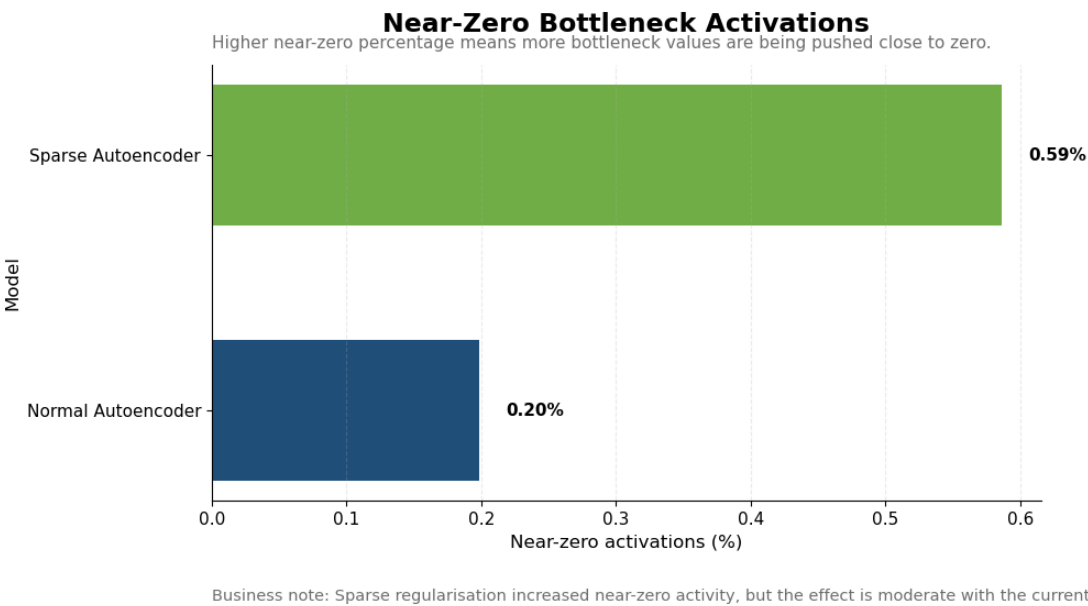
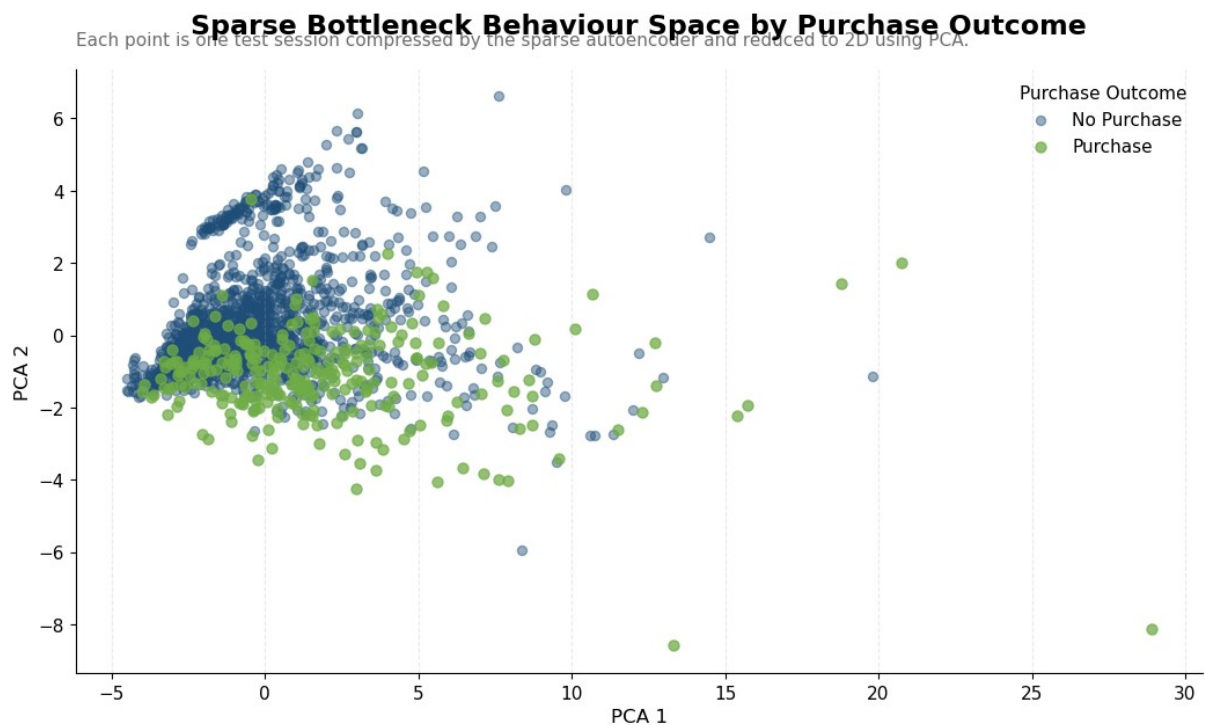


Figure 8. Near-zero activations. Sparse AE pushes more bottleneck values closer to zero, but the effect is moderate.

5.2.1 Bottleneck Compactness Interpretation

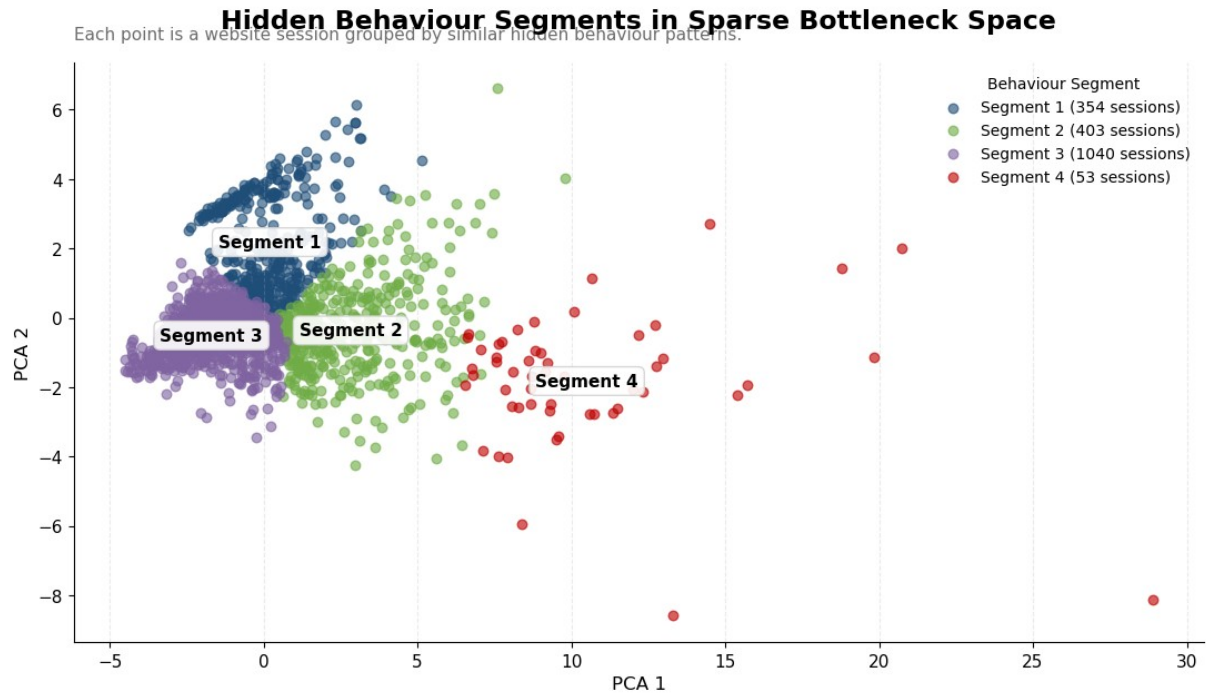
Result	Simple interpretation
Average activation dropped from 2.4879 to 1.7627	The sparse model uses the bottleneck less strongly.
Near-zero activations increased from 0.20% to 0.59%	More hidden values are closer to zero.
Sparsity is moderate	The L1 value was small. Stronger L1 could create more sparsity but may increase error.

5.3 Hidden Behaviour Space



Business note: Separation between colours would suggest different hidden behaviour patterns for buyers and non-buyers.

Figure 9. Sparse bottleneck PCA by purchase outcome. Purchase and non-purchase sessions overlap, but purchase sessions show some different spread.



Business note: These segments are created from hidden behaviour features, not from the Revenue column.

Figure 10. Hidden behaviour segments. KMeans groups the sparse bottleneck features into four segments.

Segment	Sessions	Session Share	Purchase Rate	Avg. Reconstruction Error	Business interpretation
Segment 1	354	19.14%	1.70%	0.028	Low-intent behaviour.
Segment 2	403	21.78%	32.26%	0.057	Strong practical opportunity.
Segment 3	1,040	56.22%	12.02%	0.039	Common browsing behaviour.
Segment 4	53	2.87%	47.17%	0.133	Rare, unusual, high-converting behaviour.

5.4 Business Questions Answered with BI Charts

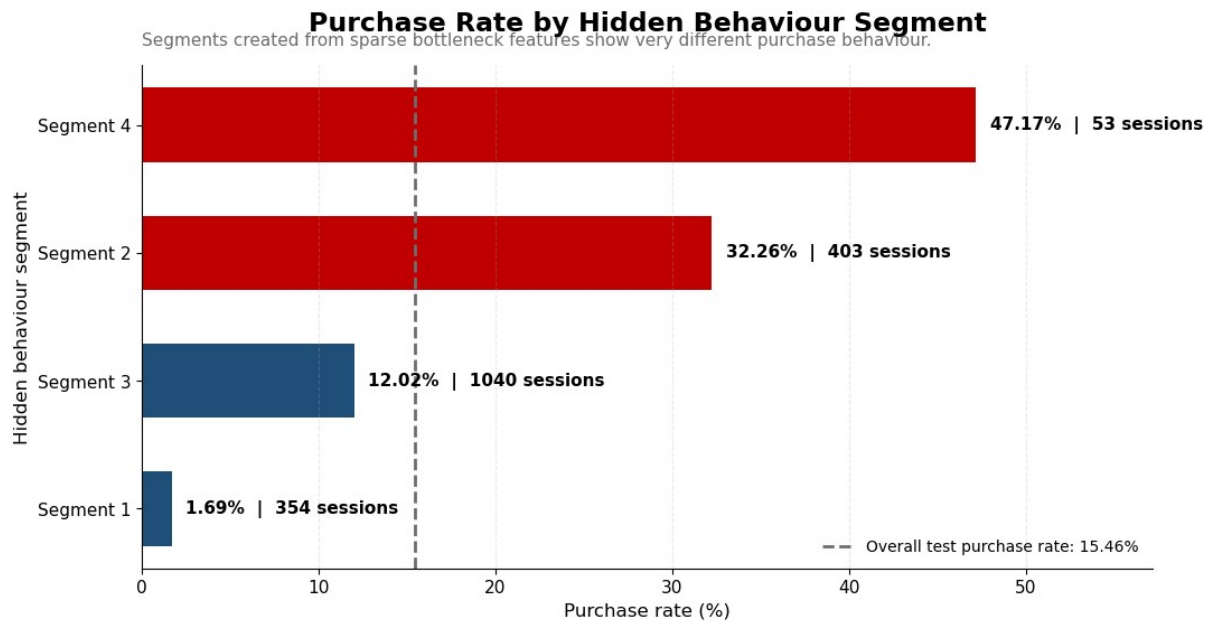


Figure 11. Purchase rate by hidden behaviour segment. Segment 4 has the highest rate; Segment 2 is more actionable due to higher volume.

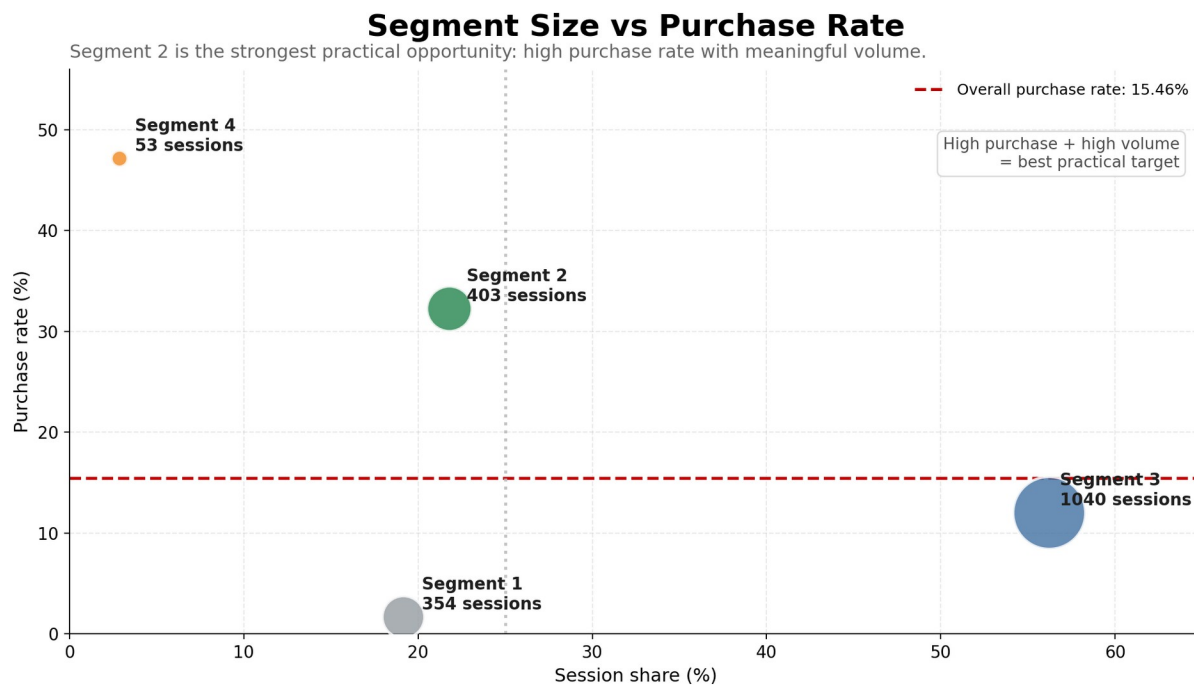
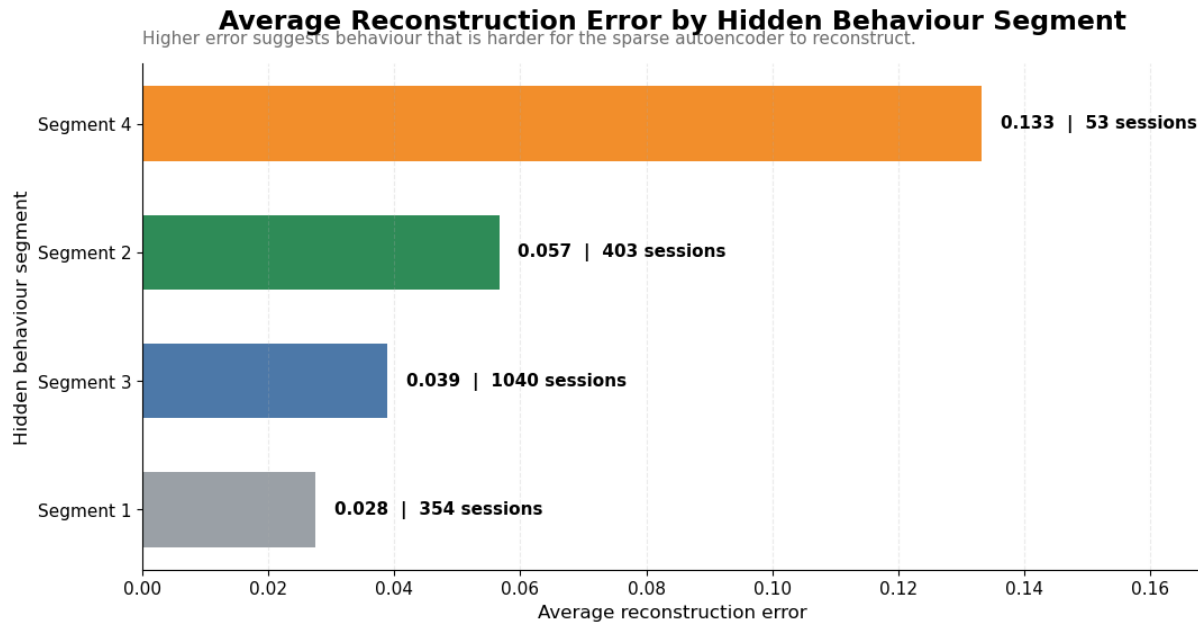
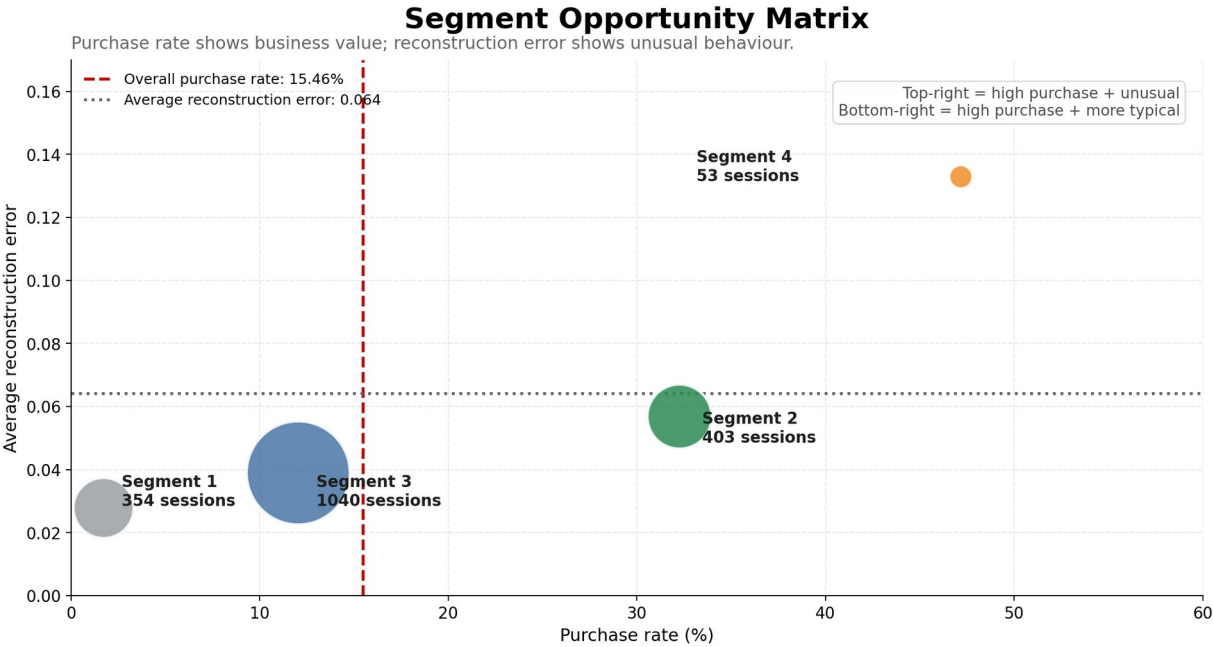


Figure 12. Segment size vs purchase rate. Segment 2 combines high conversion with useful session volume.



Business note: Segment 4 has the highest reconstruction error, suggesting rare or unusual session behaviour.

Figure 13. Average reconstruction error by segment. Segment 4 is hardest to reconstruct and may represent unusual behaviour.



Business note: Segment 2 is the strongest practical opportunity; Segment 4 is rare, high-converting, and unusual.

Figure 14. Segment opportunity matrix. Segment 2 is the strongest practical opportunity; Segment 4 is rare and unusual.

5.5 Business Use Cases Supported by Sparse Bottleneck Features

Business use case	Notebook evidence	Business reading
Customer segmentation	KMeans created 4 groups from sparse bottleneck features.	Sessions can be grouped by hidden behaviour, not just raw columns.
Purchase behaviour analysis	Purchase rate varies from 1.70% to 47.17% by segment.	Segments behave differently commercially.
Anomaly-style investigation	Segment 4 has highest reconstruction error and small volume.	Some rare behaviours may deserve deeper investigation.
Feature engineering	Each session has 12 bottleneck values.	These can become compact features for future models.

6. Limitations and Next Steps

Limitation	Why it matters	Possible next step
The model is not a classifier	Revenue was used only for interpretation.	Train a classifier later using original + bottleneck features.
Sparsity is moderate	Near-zero activations increased but stayed low.	Try stronger L1 values and compare reconstruction trade-off.
Segment labels are technical first	KMeans segment numbers do not automatically mean business labels.	Profile each segment using original columns like PageValues and BounceRates.
PCA is only a 2D view	It does not show all 12 bottleneck dimensions.	Use PCA together with tables and business metrics.
Test sample only	Segments were analysed on test data.	Repeat on full data and validate over time.

6.1 Recommended Next Analysis

1. Profile Segment 2 and Segment 4 using original business features such as PageValues, ProductRelated_Duration, BounceRates, ExitRates, Month, and VisitorType.
2. Try L1 values such as 1e-5, 1e-4, 1e-3, and compare reconstruction error vs sparsity.
3. Use the 12 bottleneck values as features in a supervised Revenue prediction model.
4. Use reconstruction error to flag unusual sessions and inspect whether they are valuable or problematic.
5. Test different numbers of KMeans segments and compare business usefulness, not only technical cluster quality.

7. Interview-Style Explanation

Short answer

I used a sparse autoencoder to learn compact hidden behaviour features from online shopping sessions. The sparse model reconstructed slightly worse than the normal autoencoder, but created more compact bottleneck features. Those features were then used to create behaviour segments with different purchase rates and unusual-behaviour levels.

A polished interview explanation:

In this project, I used an online shoppers behaviour dataset to build a sparse autoencoder. The goal was not direct purchase prediction. The goal was to learn compact hidden behaviour features from website-session data. I first built a normal autoencoder as a baseline, then built a sparse autoencoder using L1 activity regularisation in the bottleneck. The normal autoencoder reconstructed the data more accurately, but the sparse autoencoder learned a more compact representation with lower bottleneck activation. I then used the sparse bottleneck features for PCA visualisation and KMeans segmentation. The hidden segments showed different purchase rates, session volumes, and reconstruction errors. Segment 2 was the strongest practical marketing opportunity, while Segment 4 was rare, unusual, and high-converting. This showed how sparse autoencoders can support customer segmentation, purchase behaviour analysis, anomaly-style investigation, and feature engineering in digital marketing analytics.

8. Glossary of Important Terms

Term	Simple meaning	Notebook example
Autoencoder	A model that learns to rebuild its input.	Rebuilds processed website-session features.
Sparse autoencoder	Autoencoder encouraged to use fewer active hidden values.	Uses L1 activity regularisation at the bottleneck.
Encoder	Part of the model that compresses input.	Turns 28 features into a hidden representation.
Decoder	Part of the model that rebuilds input.	Turns bottleneck values back into 28 reconstructed features.
Bottleneck	Small middle layer that stores compressed information.	12 values per website session.
Activation	Neuron output value.	Average bottleneck activation fell to 1.7627 in the sparse AE.
Near-zero activation	A hidden value close to zero.	Used to measure sparse behaviour.
L1 activity regularisation	Penalty that discourages high neuron activity.	Added to sparse_bottleneck layer.
Reconstruction error	Difference between original input and reconstructed output.	Segment 4 has highest average reconstruction error.
MSE	Mean squared error; average squared difference.	Used as training loss and reconstruction error.
StandardScaler	Scales numeric values to similar range.	Used on numeric and boolean features.
OneHotEncoder	Turns text categories into numeric columns.	Used for Month and VisitorType.
ColumnTransformer	Applies different preprocessing steps to different columns.	Combines scaling and one-hot encoding.
PCA	Reduces many features into fewer visual dimensions.	Turns 12 bottleneck values into 2D chart.
KMeans	Groups similar rows into clusters.	Creates 4 hidden behaviour segments.
Segment	A group of similar sessions.	Segment 2 is high-value and actionable.
Session share	How large a segment is.	Segment 3 has 56.22% of test sessions.
Purchase rate	Percentage of sessions ending in purchase.	Segment 4 has 47.17% purchase rate.
Anomaly-style behaviour	Behaviour that is unusual or harder to reconstruct.	Segment 4 is small and has high reconstruction error.
Feature engineering	Creating useful new input features.	Use the 12 bottleneck values in future models.
Data leakage	When test information influences training.	Avoided by fitting preprocessing only on training data.
Stratified split	Split that keeps class ratio similar.	Revenue ratio stays about 84.5% / 15.5% across sets.

9. Final Master Document Takeaway

Final takeaway

For pure reconstruction, the normal autoencoder is better. For compact hidden behaviour features and business interpretation, the sparse autoencoder is valuable. In this notebook, the sparse bottleneck features helped move from raw website-session data to actionable behaviour segments.

Question	Answer
Did the sparse autoencoder reconstruct best?	No. The normal autoencoder had lower reconstruction error.
Did the sparse autoencoder create a more compact representation?	Yes. It reduced average bottleneck activation and increased near-zero activations.
Was the bottleneck useful for business analysis?	Yes. It supported hidden segmentation and purchase-behaviour comparison.
Most actionable segment?	Segment 2. High purchase rate with meaningful volume.
Rare but interesting segment?	Segment 4. Highest purchase rate and highest reconstruction error, but small.